Query-Aware Photo Selection for Data Reduction

Luca Foresti

Matricola: 565562

Roma Tre University

Report created June 25, 2026

Abstract

This project studies a practical data-reduction task from the course assignment [3]: given photo embeddings and a historical query log, choose a budgeted subset of photos to retain while preserving query usefulness. I implement the three required baselines, exhaustive cosine search (A), IndepDF query-membership scoring (B), and exact Shapley ranking (C), and propose Method D, a query-weighted greedy facility-location method. The private dataset has 41,620 photos, 2,048 embedding dimensions, and 443 query rows. Exact methods are useful only on tiny samples, Method B is the fastest scalable baseline, and Method D gives the strongest scalable cosine-proxy utility while remaining feasible on the full dataset.

1 Problem and Data Contract

The assignment gives a photo matrix `photos.csv` and a query log `queries.csv`. Each photo $d \in \mathcal{D}$ is an embedding vector, and each query row lists photo identifiers returned by one historical search. The objective is to retain at most B photos:

$$\mathcal{D}'^{*} = \underset{\mathcal{D}' \subseteq \mathcal{D},\ |\mathcal{D}'| \le B}{\arg\max}\ \mathbb{E}_{q \sim \mathcal{Q}} \left[S(q(\mathcal{D}), \mathcal{D}') \right]. \tag{1}$$

The original query operators are unavailable, so the implementation uses historical results as empirical evidence of what should remain useful. I report two metrics. The primary metric is a cosine replacement proxy:

$$S(q(\mathcal{D}), \mathcal{D}') = \frac{1}{|q(\mathcal{D})|} \sum_{d \in q(\mathcal{D})} \max_{s \in \mathcal{D}'} \cos(d, s). \tag{2}$$

The secondary metric is membership precision, the fraction of historical query results directly retained.

The loader treats the assignment photo ID as the line number in `photos.csv`, so canonical raw-data runs use one-based query IDs and then normalize to zero-based internal IDs. CSV validation rejects malformed photo rows, nonnumeric values, missing values, empty query rows, duplicate query IDs, and out-of-range IDs. In the validated private dataset, the query log references 10,937 distinct target photos after normalization.

2 Methods

All methods share the same selection interface, budget validation, normalized photo matrix, query rows, deterministic lower-ID tie-breaking, timing, memory measurement, and diagnostics. Exact methods return a documented `skipped` result when a configured guardrail is exceeded.

Table 1: Implemented methods and their role in the comparison.

Method	Signal	Optimization	Role
A	Cosine proxy	Exhaustive subset search	Exact tiny baseline
B	Query membership	IndepDF score ranking	Scalable query baseline
C	Cosine proxy	Exact Shapley ranking	Principled tiny baseline
D	Query weights + cosine	Greedy facility location	Proposed scalable method

Method A: exhaustive cosine search. Method A enumerates all size-B subsets and returns the subset with maximum cosine-proxy utility. It is the cleanest correctness reference on tiny samples, but its $\binom{m}{B}$ search space makes it infeasible on the full collection.

Method B: IndepDF query scoring. Method B scores each photo by its average normalized membership mass:

$$\text{score}(d) = \mathbb{E}_{q\sim\mathcal{Q}}\left[\frac{I_q(d)}{|q(\mathcal{D})|}\right], \tag{3}$$

where $I_q(d) = 1$ if d appears in query result $q(\mathcal{D})$ and 0 otherwise. It is extremely fast and memory-light, but it does not use embedding similarity or diversity.

Method C: exact Shapley ranking. Method C computes the exact Shapley value of each photo under the cosine-proxy utility and selects the top B photos [2]:

$$\phi_d = \sum_{T\subseteq\mathcal{D}\setminus\{d\}} \frac{|T|!(m-|T|-1)!}{m!}\left(f(T\cup\{d\}) - f(T)\right). \tag{4}$$

This gives a principled individual contribution score, but exact coalition enumeration is exponential, so the implementation only runs it on small samples.

Method D: query-aware facility location. Method D is the proposed method. It follows the workload-aware data-forgetting perspective of retaining data that preserves downstream utility [1]. It assigns each photo a query-derived weight w_d using the same normalized query mass as Method B, then greedily maximizes weighted clipped cosine coverage:

$$F(S) = \sum_{d\in\mathcal{D}} w_d \max_{s\in S} \max(0, \cos(d, s)). \tag{5}$$

At each step it maintains the best current similarity for every positively weighted target and selects the candidate with the largest marginal gain. Negative cosines are clipped to zero so that a retained photo can only provide positive replacement evidence. Candidate evaluation is chunked by a configurable memory limit, so Method D can run on the full dataset without building a dense all-pairs similarity matrix.

Table 2: Cost drivers. Here m is candidate photos, r total query memberships, t positively weighted targets, k budget, and c the Method D candidate chunk size.

Method	Main time cost	Main memory cost
A	$\mathcal{O}\left(\binom{m}{k}\cdot\text{eval}\right)$	selected subset and evaluation buffers
B	$\mathcal{O}(r + m\log m)$	score vector of length m
C	$\mathcal{O}(2^m\cdot m\cdot\text{eval})$	coalition values and Shapley scores
D	$\mathcal{O}(k\cdot m\cdot t)$ similarities, chunked	$\mathcal{O}(t\cdot c)$ similarity chunk

3 Final Answer

The smallest assignment budget is $B = 3$. The full-data Method D run is the recommended retained set because it uses both historical query importance and embedding replacement quality.

Table 3: Recommended full-data output for $B = 3$. Normalized IDs are zero-based internal IDs; assignment line IDs are one-based photo IDs.

Method	Selected photos	Cosine proxy	Runtime (s)	Peak MB
D	normalized: 5974, 39809, 24228; line IDs: 5975, 39810, 24229	0.369236	31.172	427.795

This set should be read as the output of the implemented objective, not as a universal statement that these are intrinsically the best personal photos. If a deployment needed an immediate low-resource

answer, Method B is the fallback; for offline selection under storage pressure, Method D is the stronger choice.

4 Experiments

Experiments are driven by YAML configs and saved as tidy CSV rows plus diagnostics JSON. Figures are regenerated from saved result files rather than manually edited spreadsheets. The canonical evidence uses the private raw dataset with one-based query IDs.

Table 4: Canonical evidence batches used for the report.

Batch	Purpose	Methods
small exact comparison	compare all methods where exact A/C are feasible	A, B, C, D
scalability	scale B and D from 1,000 photos to the full dataset	B, D
budget sensitivity	test budgets 1, 3, 5, 10, and 25 on 10,000 photos	B, D
query holdout	train on 75% of projected queries and evaluate on 25%	B, D
exact infeasibility	document full-data skips for exact A and C	A, C

Exact methods. On the full dataset, Method A would need to evaluate 12,014,997,156,340 subsets for $B = 3$, and 125,007,034,163,850,445 subsets for $B = 4$. Method C is even less feasible because exact Shapley values require coalition enumeration. The code therefore skips full-data A and C with explicit diagnostics rather than silently omitting them.

Small exact comparison. On query-active samples small enough for exact search, Method D nearly matches Method A under the cosine proxy. For $B = 3$, average cosine utility is 0.612920 for A and 0.612436 for D. For $B = 4$, it is 0.696715 for A and 0.696319 for D. This is evidence that the greedy coverage objective recovers the exact coverage structure on feasible instances.

Scalability and quality. Method B is consistently fastest and lightest. On the full dataset at $B = 3$, B runs in 0.022 seconds with 1.278 MB peak measured memory, but its cosine-proxy utility is 0.283321. Method D runs in 31.172 seconds with 427.795 MB peak measured memory and reaches 0.369236 cosine-proxy utility. Averaged over the scalability batch, B reaches 0.293034 while D reaches 0.366595.

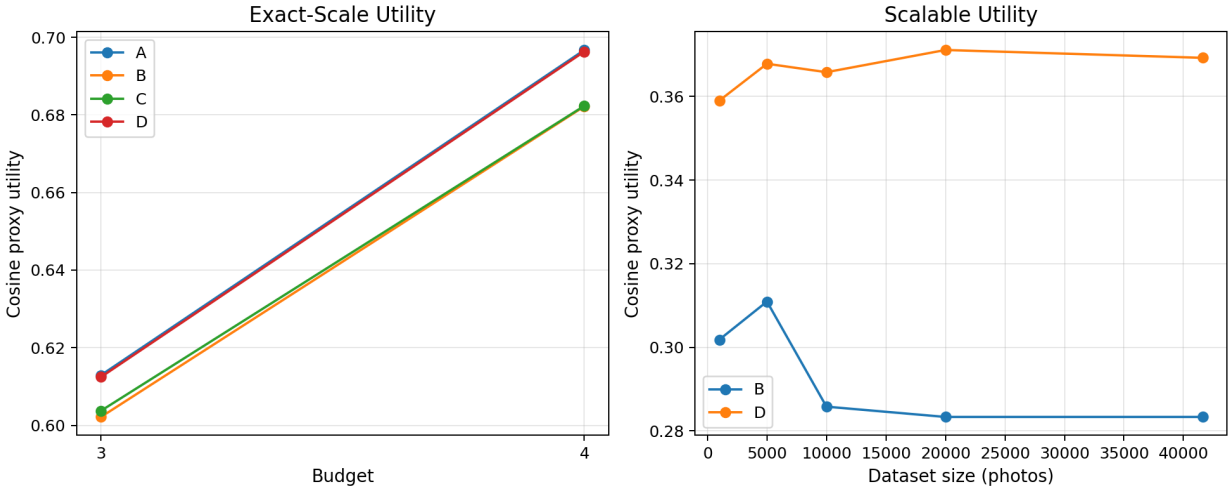

Figure 1: Cosine-proxy utility. Method D tracks exact search on tiny samples and improves over Method B on scalable runs.

Budget sensitivity and holdout. On the 10,000-photo budget-sensitivity batch, D improves from 0.293731 at $B = 1$ to 0.471688 at $B = 25$, while B improves from 0.218811 to 0.431440. In the

Table 5: Scalability rows for the two full-data-capable methods at $B = 3$.

Method	Photos	Cosine proxy	Runtime (s)
B	1,000	0.302	0.012
B	5,000	0.311	0.015
B	10,000	0.286	0.017
B	20,000	0.283	0.021
B	41,620	0.283	0.022
D	1,000	0.359	0.229
D	5,000	0.368	2.205
D	10,000	0.366	7.240
D	20,000	0.371	15.178
D	41,620	0.369	31.172

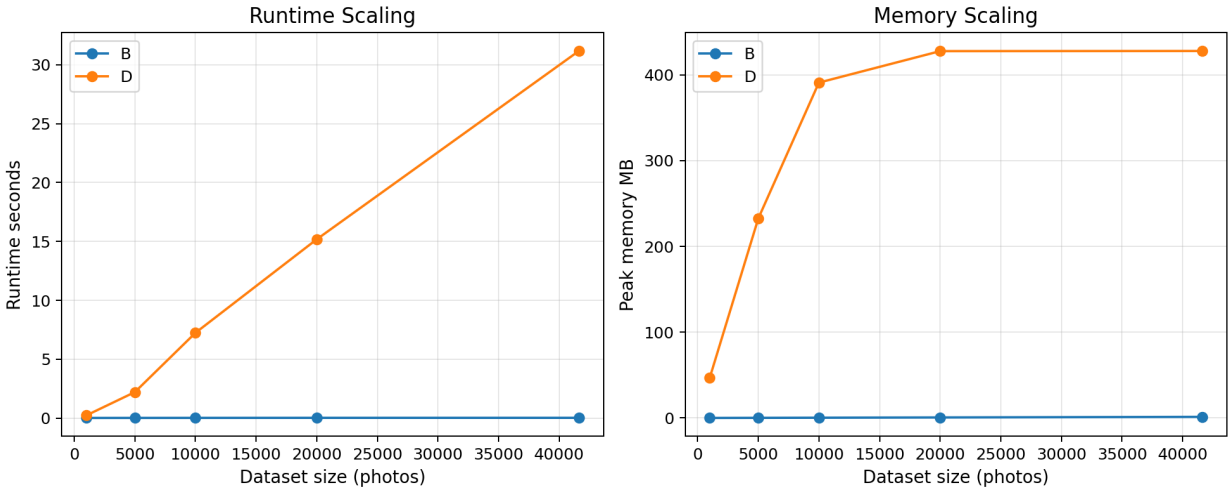

Figure 2: Runtime and memory scaling for Methods B and D. Method D is more expensive but remains feasible on the full dataset through chunked similarity coverage.

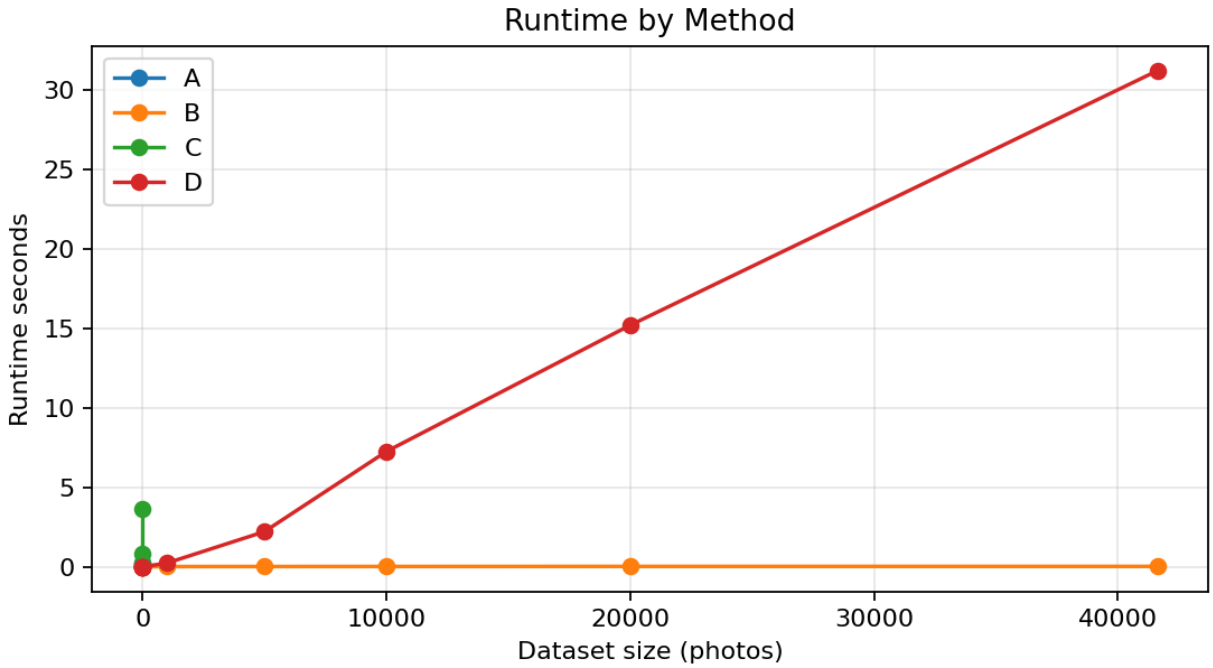

Figure 3: Runtime by method and dataset size. Exact methods appear only on tiny samples; B is effectively instantaneous, while D pays for embedding coverage.

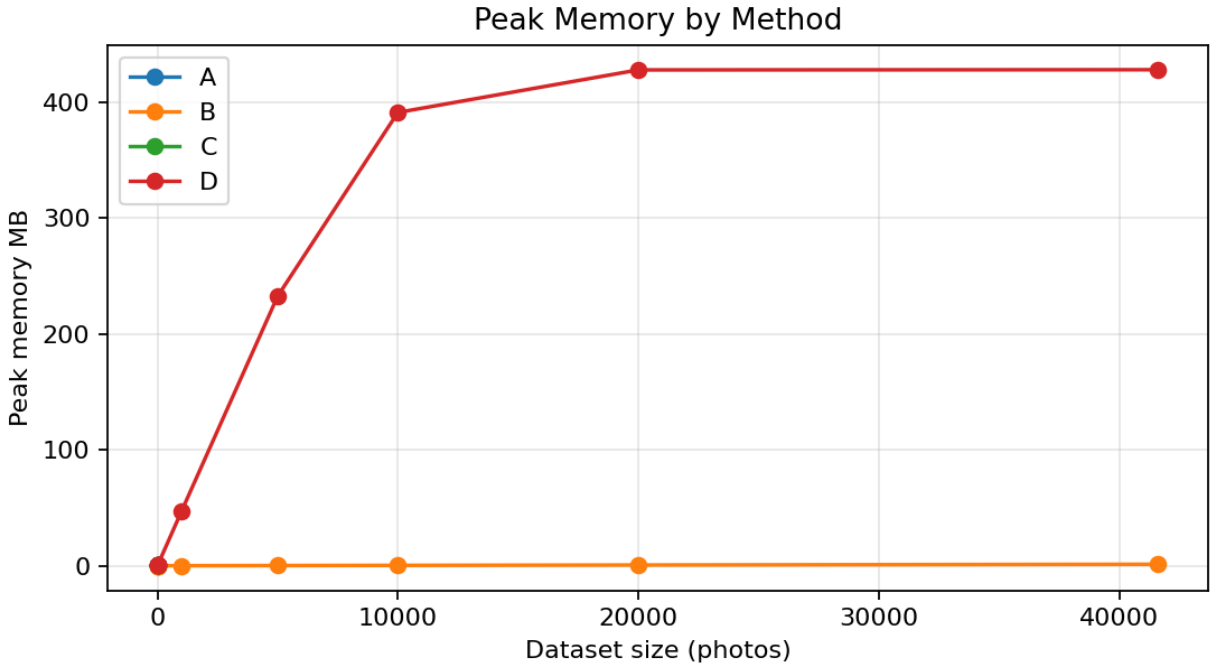

Figure 4: Peak measured memory by method. D uses more memory than B because it evaluates chunked target-candidate similarities.

query-holdout batch, averaged over budgets and seeds, held-out cosine-proxy utility is 0.399460 for D and 0.295733 for B. The holdout split is random because the query log has no timestamps, but it reduces the risk that D only fits the exact query rows used for scoring.

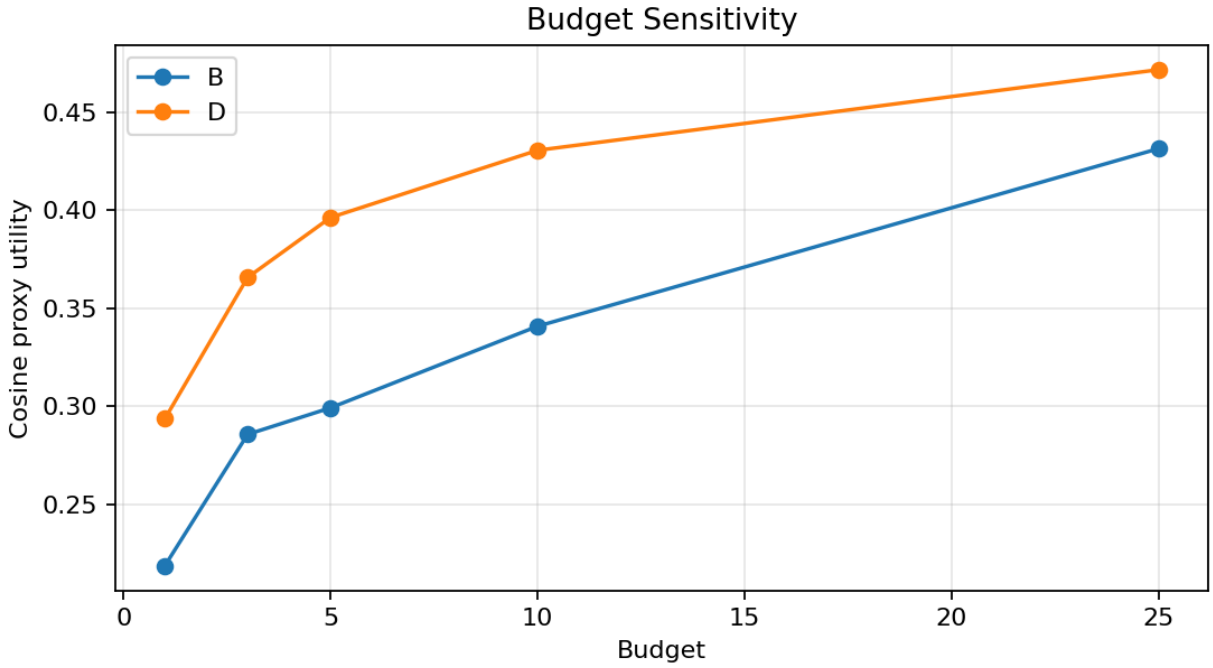

Figure 5: Budget sensitivity on a 10,000-photo query-active sample. D gains more embedding-level utility as the retention budget grows.

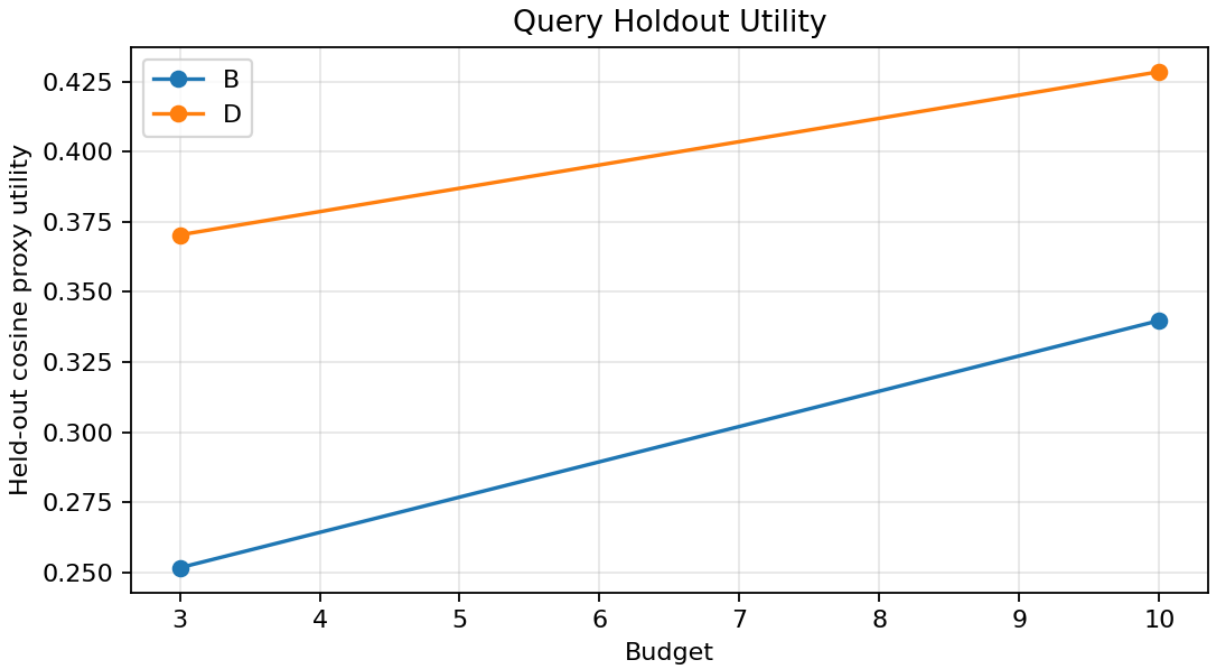

Figure 6: Held-out query utility. D remains stronger than B when methods train on 75% of projected query rows and evaluate on the remaining 25%.

5 Discussion

The methods expose a clear cost-quality trade-off. Method A is the exact cosine-proxy optimum on tiny samples but cannot scale. Method B is a strong engineering baseline because it is simple, deterministic, and essentially instantaneous, but it only preserves direct historical memberships. Method C is conceptually useful as a game-theoretic individual contribution baseline, but exact computation is not practical at assignment scale. Method D is the best compromise: it uses query history to decide which regions matter and cosine coverage to avoid wasting budget on redundant photos.

Table 6: Overall comparison.

Method	Strength	Weakness	Scale	Recommendation
A	exact under chosen proxy	combinatorial search	tiny only	correctness reference
B	fastest, lowest memory	no embedding coverage	full data	emergency fallback
C	principled contribution score	exponential exact version	tiny only	diagnostic baseline
D	best scalable cosine utility	slower and heavier than B	full data	final offline method

The main limitation is that cosine-proxy utility is not the true user utility. It is a reproducible proxy made necessary by the absence of original query operators. The second limitation is workload drift: future searches may differ from historical searches. A production mobile system should also support user constraints such as favorites, albums, and recent photos. Within the assignment setting, however, Method D is the most appropriate final strategy because it is workload-aware, embedding-aware, deterministic, and feasible on the full private dataset.

6 Reproducibility

The repository is a Python 3.12 project managed with `uv`. Raw data is intentionally ignored by git; a clean clone requires placing `photos.csv` and `queries.csv` under `data/raw/`. Core verification commands are:

```
uv sync
uv run python scripts/validate_data.py --id-base one
uv run pytest
uv run ruff check .
uv run python scripts/generate_figures.py \
  --results experiments/results \
  --output experiments/figures
```

The automated tests cover CSV validation, ID normalization, invalid data, cosine similarity, utility metrics, method determinism, exact-method skips, Method D chunking, experiment expansion, holdout evaluation, diagnostics, and figure generation.

7 Conclusion

This project implements all required methods and evaluates them under a shared, reproducible pipeline. The final recommendation is Method D with $B = 3$, retaining assignment line IDs 5975, 39810, and

24229. Method D is not exact, and it is more expensive than IndepDF, but it best matches the assignment goal: reduce the photo collection while preserving expected search usefulness from both historical query evidence and embedding-level replacement quality.

References

[1] Ramon Rico, Arno Siebes, and Yannis Velegrakis. Stochastic submodular data forgetting. In *Proceedings of the 2026 International Conference on Management of Data*, SIGMOD, 2026. Referenced in the Data Reduction Practical.

[2] Lloyd S. Shapley. A value for n-person games. In *Contributions to the Theory of Games II*, pages 307–317. Princeton University Press, 1953.

[3] Yannis Velegrakis. Data reduction practical, 2026. Course practical assignment.